

732A94 Advanced R Programming

Computer lab 5

Johan Alenlöv, Bayu Brahmantio.
(designed by Leif Jonsson, Måns Magnusson and Shashi Nagarajan)
(converted to L^AT_EX by Arash Haratian)

25 September 2026 (KY34)

Seminar date: **6 October, 10:15** (P42)
Lab deadline: **9 October 23:59**

Instructions

- Ideally this lab should be conducted by students **two by two**.
- The lab consists of writing a package that is version controlled on `github.com` or `gitlab.liu.se`.
- All students in the group should **contribute equally much** to the package. All group members have to contribute to, understand and be able to explain all aspects of the work. In case some member(s) of a group do not contribute equally this has to be reported and in this situation a formal group work contract will be signed, stipulating the consequences for further unequal contributions.
- Other significant collaborations/discussions should be acknowledged in the solution.
- To copy other's code is **NOT** allowed. Your solutions will be checked through URKUND.
- Copying solutions of others and from any online or offline resources is **NOT** allowed.
- Commit continuously your addition and changes.
- Collaborations should be done using GitHub (ie you should commit using your own github account) or using GitLab.
- In the lab some functions can be marked with an *. Students **MUST do AT LEAST ONE** exercise marked with an * for each of the Labs 3 - 6 and Bonus. If only one exercise is marked with an *, then it **MUST** be done.
- The deadline for the lab is on the lab's title page.
- The lab should be turned in using an url to the repository containing the package on `github/gitlab.liu.se` using **LISAM**. This should also include name, liu-id and, if applicable, github user names of the students behind the project. In case of problems or if you do not have access to LISAM the url may be emailed to `baybr79@liu.se` or `johan.alenlov@liu.se`.
- **NO resubmissions will be allowed for the Bonus lab.**
NO late submissions will be allowed for the Bonus lab.
- Inside your package you may not depend on any global variable (unless it is a standardR one, like `pi`). Using them will result in an immediate failure of your code. If at any stage your code changes any options, these changes have to be reverted before your code finishes.
- All notes raised by Travis/GitHub Actions/GitLab CI have to be taken care of or explicitly defended in your submission.
- The seminars are there to discuss your solutions and obtain support with problems. Every group has to present at least once during the seminars in order to pass the lab part.

Contents

1	A package connecting to a web API	3
1.1	Create a package	3
1.1.1	Write the R code	3
1.1.2	Write unit tests	3
1.1.3	Documentation and vignettes	3
1.2	* Create a Shiny application using your package	4
1.3	Seminar and examination	4
1.3.1	Examination	4

Chapter 1

A package connecting to a web API

In this lab we will create a package to connect to a web API using R. The purpose of this lab is to start the work with a API package you will use and develop further during the course. You can choose a connection to any API you find to have interesting data to analyze as well (you may use this data in a machine learning project in the last lab). If the project is good enough you will be able to submit this package to CRAN during the last week.

The API needs to be accepted by the teacher before you start. In the document `732A94_AdvancedRHT2017_Lab05_API_projects.pdf` you can find API:s suggestions to implement as an R package.

Master students will also implement a simple Shiny application using the package to connect to the API.

1.1 Create a package

Start out by creating your new package on GitHub with GitHub Actions (or Travis) or on GitLab with GitLab CI. You can choose the option you prefer. See lab 3 for details on how to set up a package.

1.1.1 Write the R code

Think of which functions you should have to access the data in the API of your choice. There are often limits and specific configurations for a given API that you need to access.

In general it is good to have as few functions as possible that are exported in the package. But behind these functions you may need to have more functionality not accessed by the users of your package.

1.1.2 Write unit tests

Write a unit test suite for the functions in the package. Examples of test can be:

1. Downloading a big query.
2. Testing the limits of the API configuration.
3. Testing the inputs and outputs of your functions.
4. Testing that the function always return a given value for a specific set of inputs.

See chapter 13, "Testing", in [3], or [2] for details.

1.1.3 Documentation and vignettes

Write a vignette that describes your package and how to use it with examples. For more information on how to write a vignette and include them in your package see chapter 17, "Vignettes", in [3].

Document all public functions, i.e., the functions you choose to export to users of your package, using `roxygen2`. See chapter 16, "Function documentation", in [3] for details.

1.2 * Create a Shiny application using your package

Create a new repository on GitHub (or GitLab) for a Shiny application. Create a simple Shiny application that makes it possible to interactively analyze data using your API package. At least one interactive widget should be used. See [1] for more details.

It should be possible to run your shiny app out using the `runGitHub()` function.

1.3 Seminar and examination

During the seminar you will bring your own computer and demonstrate your package and what you found difficult in the project.

We will present as many packages as possible during the seminar and you should

1. Show that the package can be built using R Studio and that all unit tests is passing.
2. Discuss why you implemented the functions as you did.
3. Give a short presentation of the testsuite.
4. Show your vignette/run the examples live.
5. * Present your Shiny application.

1.3.1 Examination

Turn in a the address to your github or gitlab repo with the package using LISAM. To pass the lab you need to:

1. Have the R package up on GitHub with a GitHub Actions (or Travis CI) pass/fail badge, or similarly on GitLab with a GitLab CI pass/fail badge.
2. The test suites for the implemented function(s) should be included in the package.
3. The package should build without warnings (pass) on GitHub Actions (or Travis CI, GitLab CI).
4. All issues raised by GitHub Actions / Travis CI / GitLab CI should be taken care of or justified why they are not a problem or cannot be corrected. Be careful with namespace issues, these you HAVE to take care of.

Bibliography

- [1] Posit. Shiny basics. URL: <https://shiny.posit.co/r/getstarted/shiny-basics/lesson1/>.
- [2] Hadley Wickham. testthat: Get started with testing. *The R Journal*, 3(1):5–10, 2011.
- [3] Hadley Wickham and Jennifer Bryan. *R packages*. ” O’Reilly Media, Inc.”, 2023. URL: <https://r-pkgs.org/>.